université PARIS-SACLAY

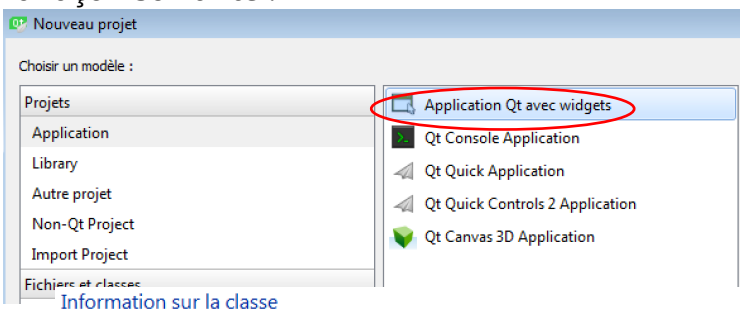
IUT DE CACHAN

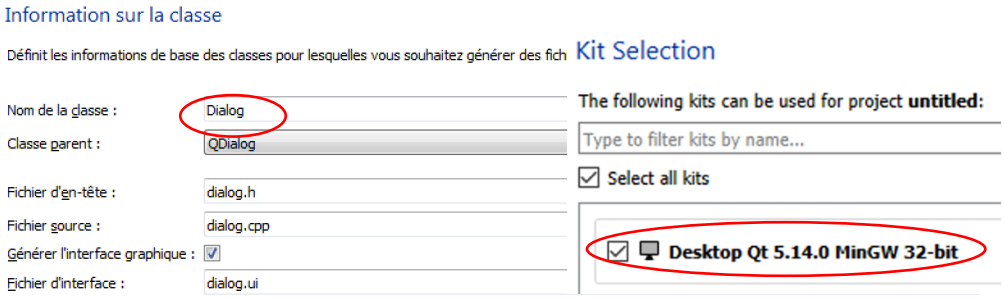
BUT3 ESE – LP MECSE
Département GEII2
Programmation C/C++

Introduction au IHM sous QT

Auteur(s) : O. Clamens
Date – Version : Nov. 2025 – v1.0
Référence : IntroIHMQT

Étape n°1 : Apprentissage=>Création d'applications graphiques sous QT.
La création d'un projet QT pour obtenir une fenêtre de dialogue souvent utilisé pour les IHM se déroule de la façon suivante :

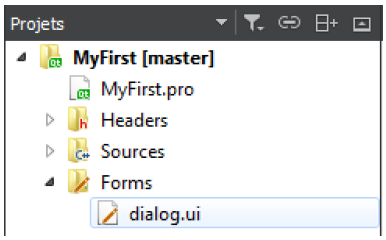




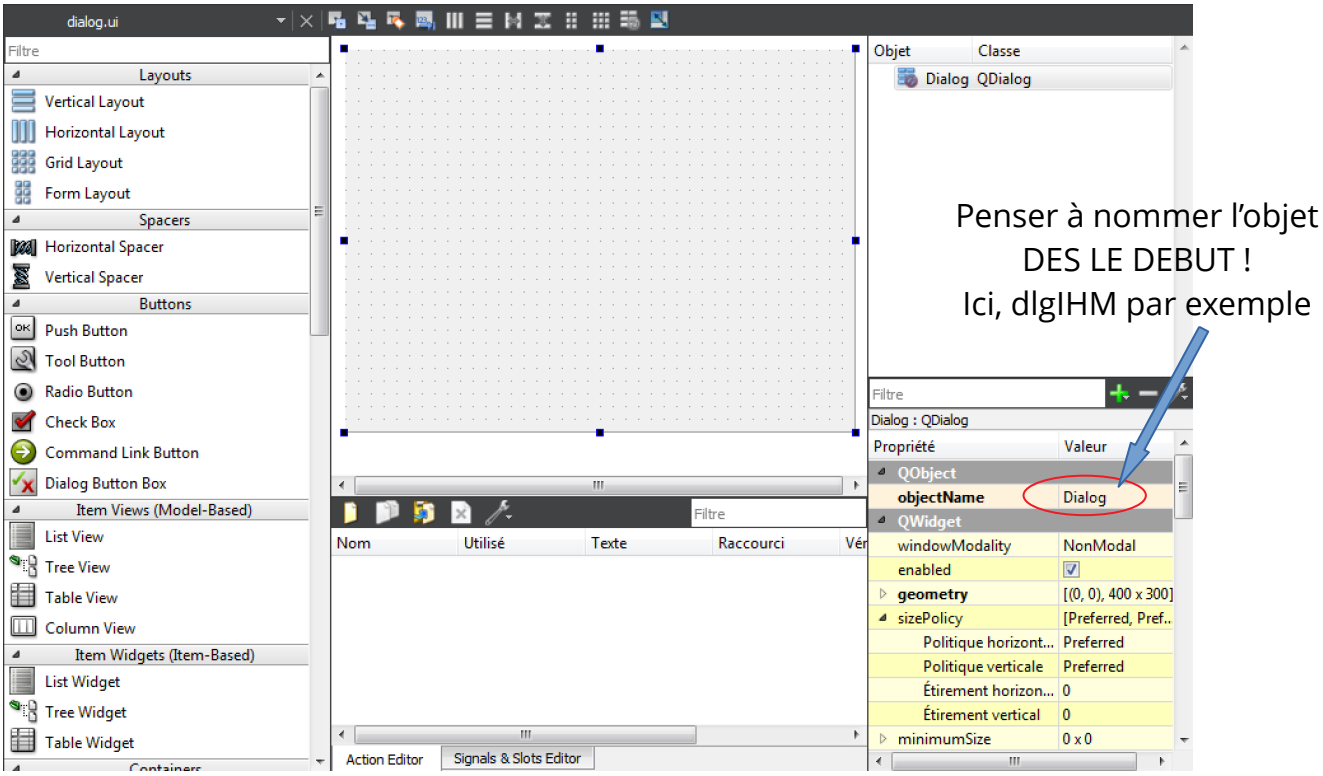


Cocher ***Desktop Qt 5.13.1***
Et Décocher ***Qt 5.13.1 WebAssembly***

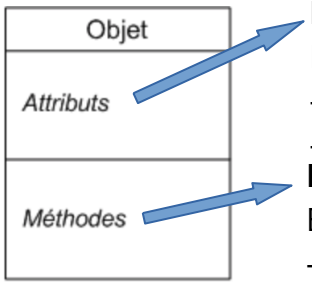
On obtient un projet de cette forme-là :



On double clic sur dialog.ui et on obtient le designer de l'IHM :



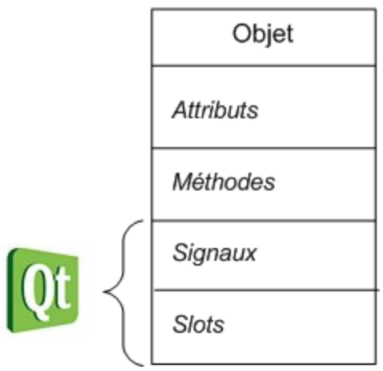
Dès lors, tous les éléments d'interface que vous allez placer sont des OBJETS :



Données caractérisant l'objet.
Exemple :
-couleur d'un objet Bouton,
-son texte

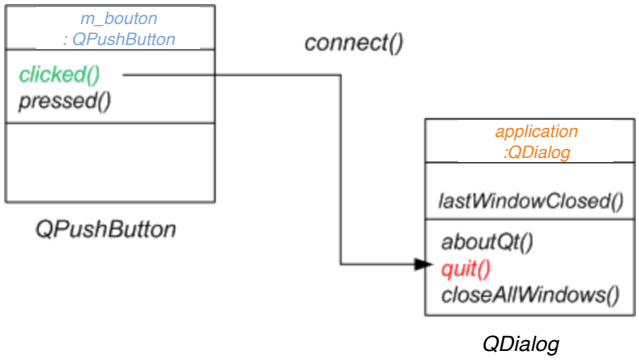
Fonctions de l'objet.
Exemple :
-Changer sa taille
-Changer sa couleur

Mais le gros intérêt, c'est de pouvoir gérer les évènements que l'utilisateur (entre autre) de l'IHM va déclencher : clic, double clic, saisie, etc....
QT permet de mettre en place cette gestion évènementielle, à l'image des interruptions, grâce au mécanisme Signal/Slot :



Un signal est un **message** envoyé par l'objet (par exemple « on a cliqué sur le bouton »). Un slot est une... méthode. En fait, c'est une méthode classique comme toutes les autres, à la différence près qu'elle a le droit d'être connectée à un signal.

Avec Qt, on dit que l'on connecte des signaux et des slots entre eux. Supposons que vous ayez deux objets, chacun ayant ses propres attributs, méthodes, signaux et slots (figure suivante). Pour simplifier, je n'ai pas représenté les attributs et les méthodes sur mon schéma.



// Connexion du clic du bouton à la fermeture de l'application
QObject::connect(ui->m_bouton, SIGNAL(clicked()), application, SLOT(close()));

REMARQUE : *m_bouton* et *application* sont les **adresses** des objets. Cette connexion doit avoir lieu généralement juste après la ligne ui→setupUi(this); de l'application

*Il faut savoir que **this** est l'adresse de l'objet où on se trouver.*

L'évènement clic sur l'objet m_bouton va déclencher l'émission de son **signal** (en vert) *clicked()*, qui, étant connecté au **slot** (en rouge) *quit()*, va déclencher automatiquement l'exécution de cette méthode.

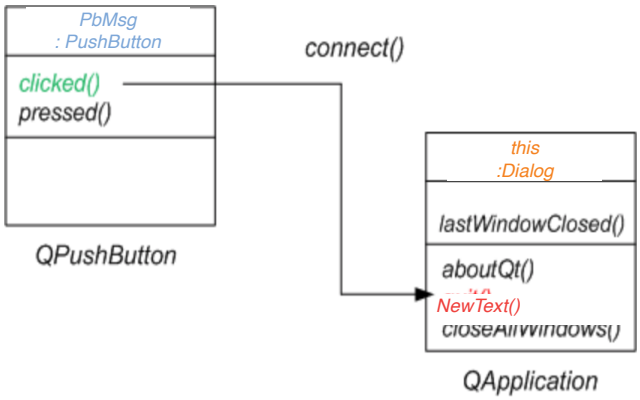
Étape n°2 : Signaux-Slots… première !

- *CRÉER une application de type QDialog, éditer le design de la fenêtre (dialog.ui) pour y placer 2 objets : un PushButton à nommer pbMsg, et un Label, à nommer lbMsg.*
- *AJOUTER, dans le fichier Dialog.h juste avant l'accolade fermante de la fin du fichier, le prototype d'une méthode de type slot :*

  private slots : void NewText(void) ;
- *DÉFINIR, comme toute bonne fonction, la fonction-membre (appelée **méthode**) dans le fichier Dialog.cpp, en ajoutant tout à la fin du fichier :*

  void Dialog ::NewText(void){
  ui->lbMsg->setText("New !") ;
  }
- *METTRE en place la connexion entre le signal et slot, juste après la ligne ui->setupUi(this) ;, en utilisant correctement* **QObject::connect(……..)**.

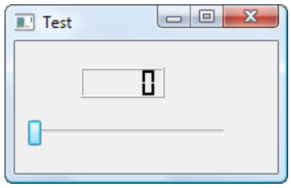
On se retrouve dès lors avec la configuration précédente :



→Valider le fonctionnement

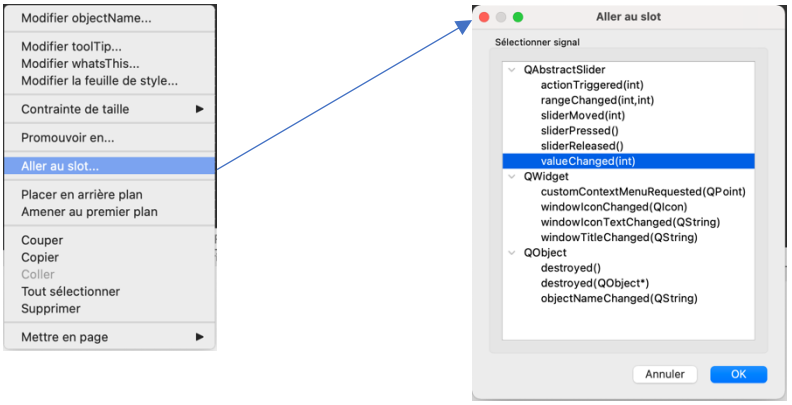
On a souvent besoin de faire passer une valeur d'un signal à un slot.

Étape n°3 : Signaux-Slots… deuxième !



Exemple : On veut qu'un Slider puisse agir sur un afficheur. Il faut donc que l'évènement "modification du curseur du slider" soit connecté à un slot de l'afficheur LCD ***et <u>lui envoi sa position</u>***.

- *CRÉER une application de type QDialog, éditer le design de la fenêtre (dialog.ui) pour y placer 2 objets : un LCDNumber à nommer lcdAff, et un HorizontalSlider, à nommer hsCurs.*
- *METTRE en place la connexion entre ces 2 objets. Cette fois-ci, nous allons utiliser l'assistant : Toujours dans le designer, faites un clic-droit sur l'objet* ***qui doit générer le signal*** *(C'est qui, ici ?) et vous obtiendrez un menu contextuel de ce type :*

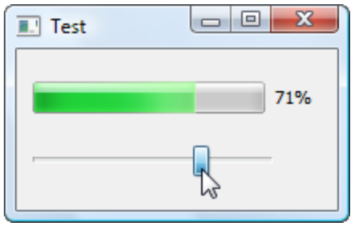


Il suffit de choisir le signal voulu.
On remarque que ce signal-là passe une valeur en paramètre de type "int"
C'est la nouvelle valeur du slider

De cette façon, Qt va créer le slot correspondant (dans le .h ET le .cpp). Il ne vous reste "plus qu'à" indiquer le code à exécuter dans ce cas là.

- *EXÉCUTER le programme et VALIDER son bon fonctionnement.*

Étape n°4 : Et… tout seul, je sais faire ?



Pour vous entraîner, je vous propose de réaliser une petite variante du code source précédent.
Au lieu d'afficher le nombre avec un QLCDNumber, affichez-le sous la forme d'une jolie barre de progression nommée Progress Bar:

Étape n°5 : Le multitâche, c'est quoi ce truc ?

Dernière exercice d'apprentissage, créer une application qui permet grâce à un bouton poussoir (PushButton) de déclencher un compteur de 0 à 5 secondes matérialisé par un LCDNumber.
Pour cela :

- *Déterminer l'esquisse de l'IHM*
- *Déterminer les objets et évènements à gérer*

Remarque : vous pouvez utiliser une "méthode" particulière pour obtenir une "pause" de durée paramétrable en ms : **QThread::msleep(int) ;**

Que remarque-t-on lors de l'exécution de ce programme ? Discuter de solutions avec votre enseignant.